CS388 NLP — Midterm Study Guide

Compiled from the course slide notes (slides-notes folder) · Prof. Greg Durrett · UT Austin MSAI
Prepared for Luis Vizcaya · October 2026

HOW TO USE THIS GUIDE

Part I covers what the past midterms (Fall 2022, Fall 2023) actually tested: **Weeks 1–8** — classification, neural nets, word embeddings, language modeling, Transformers, decoding, pre-training/seq2seq, and structured prediction (tagging & parsing). Each section gives the concepts, the formulas, a worked example, and the traps that show up on the exam. Part II is a compact review of the later-semester notes (LLMs, explanations, QA, MT, summarization, multilinguality, grounding, ethics) for completeness. Part III is an exam-day cheat sheet. Full worked solutions to both past exams are in separate PDFs in the same folder.

What the exam looks like (from FA22/FA23): 150 minutes, open-book take-home, no AI tools. ~50% multiple choice / short answer ("select all that apply" gets partial credit), ~50% long answer with hand computation. Recurring problem types:

- Run **perceptron** by hand for one epoch (watch the tie rule: is score 0 positive or negative?).
- Compute **skip-gram** softmax distributions from given word/context vectors; dot product vs cosine similarity.
- **n-gram LM** probabilities from count tables; effect of adding data.
- **Beam search / sampling / nucleus sampling** on a tiny bigram table.
- **HMM** sequence probabilities, counting valid tag sequences, **Viterbi** complexity.
- **PCFG**: extract grammar from trees, parse probability, Markovization, CKY reasoning.
- **Transformer** shapes (attention map, LM outputs) and parameter counting.
- Subword tokenization (greedy segmentation), NER with BIO tags + span precision/recall.

Contents

1. Linear classification & features
2. Perceptron
3. Logistic regression & optimization
4. Multiclass classification
5. Fairness in classification
6. Feedforward neural networks & backprop
7. Training tricks (init, dropout, Adam)
8. Word embeddings (word2vec, GloVe, fastText)
9. Bias in embeddings; DANs
10. n-gram language models & smoothing
11. LM evaluation (perplexity)
12. Neural LMs & RNNs
13. Attention & self-attention
14. Multi-head attention & positional encoding
15. Transformer architecture & parameter counts
16. Transformer LMs, causal masks, shapes
17. Decoding: greedy, beam, sampling, nucleus
18. Subword tokenization (BPE, WordPiece)
19. Pre-training: ELMo, BERT, RoBERTa
20. Seq2seq: encoder-decoder, BART, T5
21. POS tagging & sequence labeling
22. HMMs: estimation, Viterbi
23. CRFs, forward-backward, NER/BIO
24. Constituency parsing, PCFGs, CKY
25. Markovization & lexicalization
26. Dependency parsing & transition-based parsing
27. Part II: post-midterm topics
28. Part III: cheat sheet

Part I · Week 1 — Classification

1. Linear binary classification

Input text $\bar{x}$ (a string), label $y \in \{-1, +1\}$. A **feature extractor** maps the string to a vector $f(\bar{x}) \in \mathbb{R}^d$. A **weight vector** $w \in \mathbb{R}^d$ defines the classifier:

$$\hat{y} = +1 \text{ if } w^\top f(\bar{x}) \geq 0, \text{ else } -1$$

The decision boundary $w^\top f(x) = 0$ is a hyperplane. A bias b can be folded in as a constant feature (the course usually ignores the bias).

Feature extraction (sentiment example)

- **Bag-of-words (BoW)**: one dimension per vocabulary word (e.g., 10,000), value = count or 0/1 presence. "the movie was great" → 1s at *the, movie, was, great*, 0 elsewhere. Word order is lost.
- **Bag-of-n-grams**: features for consecutive sequences, e.g., bigrams *the movie, movie was, was great*. Captures local order like "not good". Feature space grows a lot.
- **tf-idf**: $\text{tf} \times \text{idf}$, tf = count of term in document, idf = $\log \frac{\#\text{docs}}{\#\text{docs containing } w}$. Down-weights words that appear everywhere (*the*). It reweights values; it does *not* change the number of features.

Preprocessing pipeline

1. **Tokenization**: "was great!" → *was great !*; "wasn't" → *was n't*.
2. (Sometimes) **stopword removal** (the, of, a).
3. (Sometimes) **casing**: lowercasing or truecasing.
4. **Unknown words**: map rare/unseen words to `UNK`.
5. **Indexing**: a map from each word / n-gram to a feature index.

> **EXAM TRAP (FA22 Q4)**
> Which steps *reduce* the size of a unigram feature space? Lowercasing (merges Great/great), stemming (merges great/greatly), removing stopwords, and splitting punctuation off tokens (great! and great become one type) all shrink it. Bigrams enlarge it. tf-idf changes values, not dimensionality.

2. Perceptron

```
for t in 1..epochs:
  for i in 1..D:
    y_pred = +1 if w·f(x_i) ≥ 0 else −1        # check the tie rule on the exam!
    if y_pred == y_i: no update
    elif y_i == +1:  w ← w + α f(x_i)
    else:            w ← w − α f(x_i)
```

- Update only on mistakes. The step size α doesn't matter for binary perceptron starting at $w = 0$ (it just rescales w).
- **Convergence**: guaranteed only if the data are **linearly separable**. Otherwise it cycles forever.
- **Perceptron as loss minimization**: it is SGD on the loss $\max(0, -y\, w^\top f(x))$. Gradient is 0 when correct and $-yf(x)$ when wrong.
- XOR-like data such as {good:+, bad:−, not good:−, not bad:+} with unigram features is **not separable**. Adding bigram features (e.g., *not_good*) makes it separable.

WORKED EXAMPLE

Features [good, bad, not, movie]. Data: (good movie, +) = [1,0,0,1]; (bad movie, −) = [0,1,0,1]; (not good, −) = [1,0,1,0]. Rule: score ≥ 0 → +. Start at $w = 0$.

1) score 0 → + ✓. 2) score 0 → + × (gold −): $w = [0, -1, 0, -1]$. 3) score 0 → + ×: $w = [-1, -1, -1, -1]$. Epoch 2: (good movie) score −2 → − ×: $w = [0, -1, -1, 0]$. (bad movie) −1 ✓. (not good) −1 ✓. Epoch 3: everything correct, so it has converged.

3. Logistic regression (LR)

A **discriminative** probabilistic model of $P(y \mid x)$ (unlike generative models such as Naive Bayes or HMMs, which model $P(x, y)$):

$$P(y = +1 \mid x) = \frac{e^{w^\top f(x)}}{1 + e^{w^\top f(x)}} = \sigma(w^\top f(x))$$

Decision: predict +1 if $P(y = +1 \mid x) > 0.5 \iff w^\top f(x) > 0$, so it is still a **linear** classifier. Training maximizes the log likelihood, which is the same as minimizing the **negative log likelihood (NLL)**:

$$\mathcal{L}(w) = -\sum_i \log P(y^{(i)} \mid x^{(i)})$$

Gradient for an example with $y = +1$: $\frac{\partial \mathcal{L}}{\partial w} = -f(x)\,(1 - P(y = +1 \mid x))$. SGD update:

$$w \leftarrow w + \alpha\, f(x)\,\big(1 - P(y = +1 \mid x)\big) \quad (y = +1) \qquad w \leftarrow w - \alpha\, f(x)\, P(y = +1 \mid x) \quad (y = -1)$$

- If the model is already confident and right ($P \to 1$), the update is about 0. If it's confidently wrong, the update is about $\pm f(x)$, like a perceptron update. **Intuition: LR is a "soft perceptron".**
- LR with BoW features is a **convex** problem: SGD finds the global optimum (with appropriate step sizes).
- On **separable** data, unregularized LR weights grow without bound (pushing probabilities toward 1). LR and perceptron do **not** converge to the same boundary, even with infinite epochs.

Optimization basics

- **SGD**: repeatedly pick one example and step $w \leftarrow w - \alpha \nabla_w \mathcal{L}(x_j, y_j, w)$.
- **Step size**: too large oscillates or diverges (e.g., on $L = w^2$ with $\alpha = 1$, w bounces between ± 1). Too small is slow. Try 1, 0.1, 0.01. Use a fixed decay schedule like $1/t$, or decrease the step size when dev performance stalls.
- **Newton's method**: $w \leftarrow w - (\nabla^2 L)^{-1} \nabla L$. Uses the inverse Hessian, which is very expensive to compute. **Adagrad, Adadelta, Adam** approximate this with per-parameter adaptive step sizes.
- **Regularization**: often not really needed for linear text models; early stopping works.

4. Multiclass classification

Two formulations:

Different weights (DW)	Different features (DF)
One weight vector per class w_y; features $f(x)$ shared. $\hat{y} = \arg\max_y w_y^\top f(x)$	One weight vector w; features depend on the class $f(x, y)$ (replicate $f(x)$ into the block for class y, or use conjunction features like [contains *drug* ∧ y=Health]). $\hat{y} = \arg\max_y w^\top f(x, y)$

Multiclass perceptron (DW): on a mistake, $w_{y^*} \mathrel{+}= \alpha f(x)$ and $w_{\hat{y}} \mathrel{-}= \alpha f(x)$. With DF: $w \mathrel{+}= f(x, y^*) - f(x, \hat{y})$.

Multiclass LR (softmax regression):

$$P(y \mid x) = \frac{\exp(w^\top f(x,y))}{\sum_{y'} \exp(w^\top f(x,y'))} \qquad \nabla_w \mathcal{L} = -f(x,y^*) + \mathbb{E}_{y \sim P(y|x)}[f(x,y)]$$

Intuition: push toward the gold features, away from what the model currently expects. When $P(y^* \mid x) = 1$ the gradient is 0.

> **WORKED EXAMPLE — SOFTMAX**
> Scores $(-1.1,\ 2.1,\ -0.4)$ → exponentiate $(0.333,\ 8.166,\ 0.670)$ → sum 9.17 → probabilities $(0.036,\ 0.891,\ 0.073)$. Softmax = "exponentiate and normalize".

Example tasks: topic classification (20 Newsgroups), textual entailment / NLI (entails / contradicts / neutral over sentence pairs — hard with BoW), entity linking (≈4.5M classes, so f(x,y) needs structure), authorship attribution (character 4-grams and word n-grams work well; "k-signatures").

Sentiment: BoW + LR/NB/SVM works decently (Pang et al. 2002; Wang & Manning 2012). It fails on negation ("not really very enjoyable") and discourse ("gross and overwrought, but I liked it"). Current state of the art uses large pretrained networks (SST-2: about 90 → 97).

5. Fairness in classification

- **Cleary (1966–68)**: a test is biased if predictions for a subgroup make consistent nonzero errors relative to the aggregate.
- **Thorndike (1971), Petersen & Novik (1976)**: the ratio of predicted positives to true positives should be about equal across groups. Using different thresholds per group can make results fairer.
- Dropping sensitive features isn't enough: **proxies** correlate with them (ZIP code ↔ race; BoW features pick up dialects like AAVE). Example: Amazon's recruiting tool penalized "women's X" organizations.
- Accuracy alone won't reveal these problems. Ask which subgroups matter and whether your features correlate with protected classes.

Part I · Week 2 — Neural Networks

6. Feedforward neural networks

Idea: instead of a linear model on $f(x)$, transform the data into a **latent feature space** where it becomes linearly separable.

$$z = g(Vf(x)), \qquad P(y \mid x) = \text{softmax}(Wz) = \text{softmax}(W\, g(Vf(x)))$$

- V: $d \times n$ (n input features → d hidden units); g: nonlinearity (tanh, ReLU, sigmoid); W: num_classes × d.
- Geometrically: a linear map (warp), a shift (bias), then a nonlinear squash. Without a nonlinearity, stacked linear layers collapse into one linear layer.
- **XOR example**: with $z = [\tanh(x_1), \tanh(x_2), \tanh(x_1 + x_2)]$, the "not bad" pattern becomes separable, because $\tanh(1) \approx 0.76$ but $\tanh(2) \approx 0.96 \neq 2 \times 0.76$.
- **Sigmoid** $\sigma(x) = 1/(1 + e^{-x})$ goes from 0 to 1. **tanh** goes from −1 to 1. ReLU = max(0, x).

Training & backpropagation

Loss = NLL of the gold label i^*: $\mathcal{L} = -\log P(y = i^* \mid x) = -Wz \cdot e_{i^*} + \log \sum_j \exp(Wz) \cdot e_j$.

- The gradient w.r.t. W looks exactly like logistic regression with z as the features.
- Gradient w.r.t. V uses the chain rule: $\frac{\partial \mathcal{L}}{\partial V_{ij}} = \underbrace{\frac{\partial \mathcal{L}}{\partial z}}_{\text{err}(z)} \underbrace{\frac{\partial g(a)}{\partial a}}_{\text{activation grad at } a} \underbrace{\frac{\partial a}{\partial V_{ij}}}_{f(x)}$, with $a = Vf(x)$.
- **Backprop** = forward pass computes activations; the backward pass combines backward gradients with forward-pass values. Automatic differentiation (PyTorch) does this for you.

```
class FFNN(nn.Module):
    def __init__(self, inp, hid, out):
        super().__init__()
        self.V = nn.Linear(inp, hid); self.g = nn.Tanh()
        self.W = nn.Linear(hid, out); self.log_softmax = nn.LogSoftmax(dim=-1)
    def forward(self, x):
        return self.log_softmax(self.W(self.g(self.V(x))))
# training loop: model.zero_grad(); loss = NLL; loss.backward(); optimizer.step()
```

> **EXAM TRAP (FA22 Q6)**
> Dimensions must chain: embedding dim = input dim of the first Linear; there must be a nonlinearity between Linear layers; output dim = number of classes; finish with softmax or log-softmax and nothing after. "50-dim emb → [50×75] → Tanh → [75×2] → log-softmax" is correct. Two Linears with no nonlinearity, or a Tanh right before the softmax, are wrong.

7. Practical training tricks

- **Batching**: process [batch, feats] at once for faster matrix ops; batch sizes of 1–100 work well. Sum or average the loss over the batch.
- **Initialization**: the problem is nonconvex, so initialization matters. **All zeros fails**: every hidden unit stays identical and the hidden layers get zero gradient. Too large saturates tanh (gradient ≈ 0). **Glorot/Xavier**: $U\left[-\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}, +\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}\right]$ keeps activation and gradient variance stable across layers.
- **Batch normalization**: rescale each layer to mean 0 and variance 1 over the batch (useful for deep nets).

- **Dropout**: randomly zero units during training and use the full network at test time. It's stochastic regularization, similar to ensembling: the network must be robust to missing signals.
- **Optimizers**: Adam (adaptive step size, the most common). Adaptive methods can sometimes generalize worse than SGD (Wilson et al. 2017). **Gradient clipping** caps exploding gradients.

Part I · Week 3 — Word Embeddings

8. Word embeddings

Distributional hypothesis (Firth 1957): "You shall know a word by the company it keeps." Words in similar contexts get similar low-dimensional vectors (d ≈ 50–300). *movie* and *film* both appear after "I watched the ___", so their vectors end up close.

Skip-gram (word2vec, Mikolov et al. 2013)

Each word type has a **word vector** v_w and a **context vector** c_w. Predict each context word within a window of k from the center word:

$$P(\text{context} = y \mid \text{word} = x) = \frac{\exp(v_x^\top c_y)}{\sum_{y' \in V} \exp(v_x^\top c_{y'})}$$

- Training pairs for "the cat ran away", window ±2: (the,cat) (the,ran) (cat,the) (cat,ran) (cat,away) (ran,the) (ran,cat) (ran,away) (away,cat) (away,ran). Every (center, context) pair is a separate example.
- "Skip-gram": with window > 1 you predict context words that skip over intermediate words.
- Parameters: $|V| \times d$ word vectors + $|V| \times d$ context vectors. Computing one probability costs $O(|V|d)$ because of the softmax normalizer.
- Objective: maximize $\sum \log P(\text{context} \mid \text{word})$. It can never drive every P to 1: the same word has many different contexts.

CBOW (continuous bag-of-words)

The reverse: predict the center word from the **sum** of its context vectors: $P(w \mid w_{-1}, w_{+1}) = \text{softmax}(W(c(w_{-1}) + c(w_{+1})))$. **Skip-gram: word → context. CBOW: context → word.**

Making it fast

- **Hierarchical softmax**: Huffman-code the vocabulary as a binary tree. $\log |V|$ binary decisions, each a size-d dot product, instead of $|V|$ dot products.
- **Skip-gram with negative sampling (SGNS)**: classify (word, context) pairs as real or fake. $P(y = 1 \mid w, c) = \sigma(w \cdot c)$. Objective: $\log \sigma(w \cdot c) + \sum_{i=1}^{k} \log \sigma(-w \cdot c_i)$ with k negatives sampled from the unigram distribution. Same parameter count; no $O(|V|)$ term.
- **Levy & Goldberg (2014)**: SGNS is implicitly a weighted factorization of the shifted PMI matrix $M_{ij} = \text{PMI}(w_i, c_j) - \log k$, where $\text{PMI}(w, c) = \log \frac{P(w,c)}{P(w)P(c)}$.

GloVe and fastText

- **GloVe** (Pennington 2014): weighted regression on the log co-occurrence matrix: $\sum f(\text{count}_{ij}) \, (w_i^\top c_j + a_i + b_j - \log \text{count}_{ij})^2$. Cost is constant in dataset size (only needs counts) and quadratic in vocabulary size.
- **fastText** (Bojanowski 2017): like SGNS, but a word is represented by the sum of its character n-gram vectors (n=3–6) plus the word itself: *<wh, whe, her, ere, re>*, ..., *<where>*. Handles rare and unseen words and morphology.
- **ELMo/GPT/BERT**: contextual embeddings over subwords; you must run the model to get a vector (no lookup table).

WORKED EXAMPLE — SKIP-GRAM DISTRIBUTION (FA23 Q3 STYLE)

$v_2 = (1, 1)$; contexts $c_1 = (0, 0), c_2 = (0, 1), c_3 = (1, 0)$. Dots: $0, 1, 1$. Take $e = 3$: $\exp = 1, 3, 3$ → $P = (1/7, 3/7, 3/7)$. A zero word vector $v_1 = (0, 0)$ gives all dots 0 → uniform $(1/3, 1/3, 1/3)$.

WORKED EXAMPLE — DOT PRODUCT VS COSINE (FA22)

good=[0,0,3], bad=[0,1,3], the=[10,10,10]. Dot products: (good,bad)=9, (good,the)=30, (bad,the)=40 → *the* wins because it's long. Cosine: 0.949, 0.577, 0.730 → (good,bad) is most similar. **Dot product rewards vector length (frequent words); cosine ignores length.** After normalizing to unit length, dot product = cosine.

9. Bias in embeddings & using embeddings

- Embeddings trained on web text encode societal biases: "man : computer programmer :: woman : homemaker".
- **Bolukbasi et al. (2016)**: find a gender direction/subspace (he–she and similar pairs), project gender-neutral words onto it, and **neutralize** them (subtract the projection). They also **equalize** pairs. The bias they target is the correlation with the he–she axis.
- **Gonen & Goldberg (2019)**, "Lipstick on a pig": debiasing is shallow. Male- and female-associated words still cluster together because bias pervades the space, not just one direction.
- **Using embeddings in a network**: (1) learn from scratch, (2) initialize with GloVe and freeze (faster), or (3) initialize with GloVe and fine-tune (often best). Intrinsic evaluation: word similarity and analogies.
- **Deep Averaging Network (DAN)** (Iyyer 2015): average the word embeddings, then a FFNN. Learnable parameters: word embeddings + feedforward layers (no attention). Simple averaging competes with syntax-aware tree models on sentiment.

KEY INTUITION

Embeddings turn "same word or not" into "similar meaning or not". That's why pretrained embeddings help generalize to unseen words (plural nouns, new slang near known negative words). Unseen-word problems are fixed by: pretrained embeddings, subwords, stemming, or more data.

Part I · Week 4 — Language Modeling & Self-Attention

10. n-gram language models

An LM places a distribution over sequences, using the chain rule:

$$P(\bar{w}) = P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2)\cdots = \prod_i P(w_i \mid w_1, \ldots, w_{i-1})$$

n-gram assumption: condition only on the previous $n-1$ words (an $(n-1)$-order Markov model). Bigram: $P(w_i \mid w_{i-1})$; trigram: $P(w_i \mid w_{i-2}, w_{i-1})$. Each context is a multinomial over the vocabulary, so a bigram LM has about $|V| \times |V|$ parameters.

MLE estimation: $P(\text{dog} \mid \text{the}) = \frac{\text{count(the dog)}}{\text{count(the)}}$.

Uses: generation, MT, grammatical error correction, and learning representations (word2vec, pre-training).

> **WORKED EXAMPLE (FA23 Q7)**
> Counts: (the,frog)=4, (frog,is)=2, (frog,was)=4, (is,UNK)=1, (is,frog)=1. $P(\text{the} \mid \langle s \rangle) = 1$.
> P(the frog is blue) = $1 \cdot \frac{4}{4} \cdot \frac{2}{6} \cdot \frac{1}{2} = \frac{1}{6}$ (blue → UNK). Normalize by the count of the *context* word (frog appears 6 times as a context).

Smoothing

Problem: MLE gives 0 to unseen n-grams, e.g., P(Austin | want to go to) = 0 if never seen, which zeroes the whole sentence.

- **Add-one / add-k (Laplace)**: add k to every count. Simple; used for HMM emissions.
- **Interpolation**: $P = \lambda P_{\text{trigram}} + (1-\lambda)P_{\text{bigram}}$, i.e., back off to shorter context.
- **Absolute discounting**: subtract a constant δ (≈0.75) from each seen count and give the reserved mass to a lower-order distribution, normalized within the context: $P_{AD}(w \mid c) = \frac{\max(\text{count}(c,w)-\delta, 0)}{\text{count}(c)} + \lambda(c)P(w)$, with $\lambda(c) = \frac{\delta \cdot |\{w : \text{count}(c,w) > 0\}|}{\text{count}(c)}$.
- **Kneser-Ney**: absolute discounting where the lower-order distribution uses **continuation counts**: how many distinct contexts a word appears in. "Francisco" is frequent but only after "San", so it gets a low continuation probability.

11. LM evaluation: perplexity

Accuracy doesn't work (predicting the next word exactly is usually impossible). Use the likelihood of held-out data, normalized per token:

$$\text{PPL} = \exp\left(-\frac{1}{n}\sum_{i=1}^{n} \log P(w_i \mid w_1, \ldots, w_{i-1})\right) \quad \text{(lower is better)}$$

> **WORKED EXAMPLE**
> Probabilities 1/4, 1/3, 1/4, 1/3 → average NLL = $\frac{\ln 4 + \ln 3}{2} = 1.242$ → PPL = $e^{1.242} = \sqrt{12} = 3.464$, the **geometric mean of the denominators**. Intuition: the model is as uncertain as a uniform choice among ~3.5 words. A uniform model over $|V|$ words has PPL $= |V|$. Typical values are 10–200.

12. Neural LMs and RNNs

- **Neural LM** (Bengio 2003; Mnih & Hinton): $P(w_i \mid w_1..w_{i-1}) = \text{softmax}(U\, f(w_1..w_{i-1}))$, where f embeds the context. If f is a DAN, it ignores word order, which is bad for LMs.
- FFNNs can't handle variable-length input: each input position has fixed semantics, so "great" at position 4 and position 3 live in different subspaces.
- **RNN**: process each token the same way while carrying context: $h_t = \tanh(Wx_t + Vh_{t-1} + b)$. LSTMs add a cell state c with gates.
- **Problems**: (1) **vanishing gradient** — the gradient shrinks through repeated tanh (≈0 outside [−2,2]) and repeated multiplication by V; (2) **slow** — $O(n)$ sequential, non-parallelizable steps; (3) the state is dominated by **recent tokens**.

> **EXAM TRAP (FA23 Q5–6)**
> A unidirectional RNN classifier reading "...The play was terrible... At least the dinner was excellent." predicts positive because the final hidden state is heavily influenced by recent tokens. BoW, DAN, and Transformers have no recency bias, so they would all do better here.

13. Attention

Attention lets a model access arbitrarily far-back context directly. **Running example**: sequences of A/B where the label is "B if any B is present". RNNs must carry that bit across many steps; attention can just "look for B".

Scores between a query q and keys k_i, softmax, then a weighted sum of values:

$$s_i = k_i^\top q, \quad \alpha = \text{softmax}(s), \quad \text{output} = \sum_i \alpha_i v_i$$

> **WORKED EXAMPLE — MAKING ATTENTION PEAKY**
> Embeddings A=[1,0], B=[0,1]; sequence A A B A; query $q = [0, 1]$ ("find Bs"). Scores 0,0,1,0 → softmax gives B weight $e/(3+e) = 0.475$, which isn't decisive. With $W^K = 10I$, keys become [10,0] or [0,10], scores 0,0,10,0 → B weight $\approx e^{10}/(3+e^{10}) \approx 0.9999$. **Scaling up keys/queries makes the softmax sharper (more one-hot).**

Variants: dot product $k^\top q$; bilinear $k^\top W q = (W^K k)^\top (W^Q q)$; additive (Bahdanau). The real Transformer uses three matrices: queries, keys, values.

Self-attention (Vaswani et al. 2017)

Every token is simultaneously a query, a key, and a value. With input embeddings E (seq_len × d):

$$Q = EW^Q,\ K = EW^K,\ V = EW^V, \qquad \text{Attention}(Q,K,V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V$$

- $A = \text{softmax}(QK^\top/\sqrt{d_k})$ is **seq_len × seq_len** (row i = distribution over what token i attends to). Output $Z = AV$ is seq_len × d_v: each row is a **weighted combination of V rows**.
- Dividing by $\sqrt{d_k}$ keeps dot products from growing with dimension, so the softmax stays smooth and gradients don't vanish.
- Output length = input length: a **contextualized encoding** for each word.
- **Complexity**: $O(n^2 d)$ per layer (quadratic in length), but $O(1)$ sequential operations and $O(1)$ path length between any two words (RNN: $O(n)$ and $O(n)$).

14. Multi-head attention & positional encoding

- Softmax attention tends to become **peaked** on one token. "I visited New ___" needs both *New* and *visited*. **Multiple heads** run independent attentions (each with its own W^Q, W^K, W^V of size $d_{model} \times d_k$, usually $d_k = d_{model}/h$). Concatenate the heads and project with W^O. Over multiple layers, information combines.
- Self-attention is **permutation-invariant**: without position information, "visited" and "ate" look the same apart from content. **Positional encodings** fix this.
  - Adding a raw integer position to embeddings: the scale grows unbounded and lengths beyond training are unseen.
  - **Learned absolute** position embeddings (BERT, GPT): a vector per position.
  - **Sinusoidal** (Vaswani): $PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(\cdot)$. Different frequencies; nearby positions get higher dot products.
  - **Relative** (T5): attention depends on the distance between tokens. **ALiBi**: adds a linear bias against attending far back, with a different slope per head. **NoPE**: causal Transformers can learn position implicitly by "counting".

Part I · Week 5 — Transformers & Decoding

15. Transformer architecture

One layer (block) = **multi-head self-attention → Add & Norm → position-wise FFN → Add & Norm**.

- **Residual connections**: layer output = input + sublayer(input). Helps gradients flow through deep stacks.
- **Layer normalization**: controls activation scale in deep networks.
- **FFN** applies to each position independently: $\mathrm{FFN}(x) = W_2\,\mathrm{ReLU}(W_1 x)$, expanding $d_{model} \to d_{ff}$ (usually $4d_{model}$) and back. **Most parameters live in the FFNs.**
- Dimensions: d_{model} (residual stream), d_k (queries/keys, smaller than d_{model}), d_v (values); W^O is $(h \cdot d_v) \times d_{model}$.
- Original paper: encoder-decoder. A **decoder-only** LM uses a triangular **causal mask**.

Parameter counting per layer (with $h\,d_k = d_{model} = d$, $d_{ff} = 4d$):
Attention: $W^Q, W^K, W^V, W^O \to 4d^2$. FFN: $W_1, W_2 \to 2 \cdot d \cdot 4d = 8d^2$. LayerNorms: about $4d$ (negligible).
Total ≈ 12d² per layer.
BERT-base: 12 layers × 12 × 768² ≈ 85M + embeddings 30k × 768 ≈ 23M → about **110M** ✓. Sinusoidal positional encodings have **0 parameters**; learned ones have max_len × d.

EXAM TRAP (FA23 Q9–11)
Removing an FFN removes about 8d² per layer, so **fewer** parameters. Moving the positional encoding (sinusoidal) changes no parameter count. Swapping 2 MHA for 2 extra FFNs (a 2-layer net with 2 FFNs/layer vs a standard 4-layer) still has **fewer** parameters: same number of FFNs, two fewer attention blocks. Odd architectural changes usually make performance **worse** on aggregate.

16. Using Transformers; Transformer LMs

- **Transducer** (tagging): a prediction per token (score per tag → softmax).
- **Classifier**: average-pool token encodings, or prepend a **[CLS]** token whose final encoding is classified. Because [CLS] attends to everything, it pools for you.
- **Sentence-pair classifier**: "[CLS] s1 [SEP] s2" for entailment. Self-attention captures the alignment between words in the two sentences.
- **Transformer LM**: $P(w \mid \text{context}) = \mathrm{softmax}(W h_i)$, with W of size vocab × d. Input = tokens, output = the same tokens **shifted by one** (input "<s> I saw the dog", targets "I saw the dog running"). Total loss = sum of NLL at each position, so you train on all positions at once.
- Without a mask, the model "cheats": position i can attend to token $i+1$. A **causal mask** puts $-\infty$ above the diagonal before the softmax:

```
mask = torch.triu(torch.ones(n, n) * float('-inf'), diagonal=1)
```

Shapes (Assignment 3, a frequent exam question)
Attention map: `[seq_len, seq_len]` (unbatched) or `[batch, seq_len, seq_len]` (or `[batch, heads, seq, seq]`).
Model outputs into the loss: `[batch, seq_len, vocab_size]` log-probs (or `[seq_len, vocab]` unbatched).
For `nn.NLLLoss`, reshape to `[batch*seq_len, vocab]` with targets `[batch*seq_len]`.
Hidden states: `[batch, seq_len, d_model]`. Q/K: `[batch, seq_len, d_k]`.

- **Scaling laws** (Kaplan 2020): loss falls predictably with compute, data, and parameters.
- **Efficient attention**: quadratic cost limits length. Performer/Linformer approximate attention (no explicit A). Longformer uses fixed sparse patterns (sliding window + global tokens) to scale to 4096 tokens.

17. Decoding strategies

LMs give $P(y_i \mid y_{<i})$; seq2seq gives $P(y_i \mid x, y_{<i})$. Generation options:

1. **Greedy**: $\arg\max$ at each step.
2. **Beam search**: keep the top-k partial sequences by total probability at each step. Good for MT/QA (constrained outputs, where you want the highest-probability answer). Better models need it less.
3. **Sampling**: draw $y_i \sim P$. Too random: the long tail derails generation.
4. **Nucleus (top-p) sampling** (Holtzman 2019): sort by probability, keep the smallest set whose mass ≥ p, renormalize, then sample.

- **Beam search degenerates** on open-ended text: repetition loops ("Universidad Nacional Autónoma de México / Universidad Nacional..."). Each word is locally likely (the model is **locally normalized**) even though the sequence is bad.
- **Pure sampling**: if 10% of the mass is junk tail, you hit the tail about once every 10 words.
- Nucleus gives decent perplexity, no repetition loops, and the best human-judged naturalness (HUSE).

WORKED EXAMPLE (FA23 Q16–18)

P(a)=.8, P(the)=.15, P(dog)=.05; P(dog|a)=.98, P(dog|the)=.84, P(a|the)=.15, ...; generate 2 tokens.
Beam=2: step 1 keep {a, the}. Step 2 candidates: a dog .784, a a .008, a the .008, the dog .126, the a .0225, the the .0015 → **{a dog, the dog}**.
Sampling: every token has P > 0 → $3^2 = 9$ sequences.
Nucleus p=0.9: step 1: a(.8) is < .9, add the → .95 ≥ .9 → {a, the}. After "a": dog .98 ≥ .9 → {dog}. After "the": dog .84 < .9, add a → .99 → {dog, a}. Sequences: a dog, the dog, the a → **3**.

18. Subword tokenization

- Word vocabularies get huge and can't handle unseen words; character models work poorly. The compromise is **subwords**: frequent words stay whole and rare words split ("re_ fueling", "un_ princi_ pled").
- **BPE** (Sennrich 2016): start from characters; repeatedly **count adjacent symbol pairs** (weighted by corpus frequency) and **merge the most frequent**. 30k merges → about a 30k vocabulary. Every string can be segmented.
- **WordPiece / unigram LM** (SentencePiece): merge or choose pieces that most improve a language model's likelihood. Produces more linguistically plausible units; some evidence it works better for pretrained Transformers (Bostrom & Durrett 2020).

WORKED EXAMPLE — BPE

Corpus {low:5, lower:2, newest:6, widest:3}. Pair counts: e+s = 9 (newest 6 + widest 3), s+t = 9, l+o = 7, ... Merge **e s → es**, then **es t → est** (9), then **l o → lo** (7), **lo w → low** (7)... A new word "lowest" → *low est*.

WORKED EXAMPLE — GREEDY SEGMENTATION (FA23 Q2)

Vocabulary {good, x, q, not, gre, at, great}. Greedy longest match left to right: goodx → good x; greatq → great q; **greqat → gre q at**. A typo in the middle destroys the word identity, so subwords help with suffix typos but not internal ones.

Part I · Week 6 — Pre-training & Seq2seq

19. ELMo, BERT, RoBERTa

- **ELMo** (2018): two unidirectional LSTM LMs (left-to-right and right-to-left) whose states are concatenated; frozen features. Each direction is computed in isolation.
- **BERT** (Devlin 2019) changes four things relative to ELMo: (1) **Transformers** instead of LSTMs; (2) a **bidirectional** masked-LM objective; (3) **fine-tune** the whole network instead of freezing; (4) **word pieces**.
- Why not bidirectional next-word prediction? A two-sided model can see the word it's predicting ("cheating"). **Masked LM** instead: mask 15% of tokens and predict them. Of the chosen tokens: 80% → [MASK], 10% → random word, 10% unchanged (so the model can't rely on [MASK] being present, since it never appears at fine-tuning time).
- **Next sentence prediction (NSP)**: "[CLS] A [SEP] B". 50% true next chunk, 50% random; predict IsNext/NotNext (later shown not very useful).
- Sizes: **Base** 12 layers, 768-d, 12 heads, 110M parameters. **Large** 24 layers, 1024-d, 16 heads, 340M. 30k WordPiece vocabulary; token + segment + position embeddings.
- **Uses**: classification from [CLS]; sentence-pair tasks (entailment, paraphrase), where BERT shines; tagging by predicting at each word piece. **Cannot generate text** naturally (it's an analysis model).
- **Fine-tuning**: 1–3 epochs, small learning rate. The biggest weight changes are near the top (routing information to [CLS]).
- **RoBERTa**: no NSP, **dynamic masking** (new masks each epoch), 160GB of data, longer training → better.
- Attention heads learn interpretable things: positional heads and syntactic heads (some approximate dependency arcs) — Clark et al. 2019.

> **EXAM TRAP (FA23 Q12)**
> What would likely improve BERT fine-tuning accuracy? Masking somewhat more than 15% (Wettig et al. 2023 found ~40% can be better for larger models). Making BERT unidirectional hurts (it loses bidirectional context). Replacing 100% of masks with random words makes the task too noisy.

20. Seq2seq: encoder-decoder, BART, T5

- Many tasks are sequence → sequence: MT, summarization, parsing linearized as a string, semantic parsing ("What states border Texas" → λx state(x) ∧ borders(x, e89)).
- $P(y \mid x) = \prod_i P(y_i \mid x, y_{<i})$, $P(y_i \mid \cdot) = \text{softmax}(W\bar{h}_i)$. The decoder has separate parameters from the encoder, and input/output vocabularies can differ.
- **Cross-attention**: decoder queries attend to encoder outputs (keys/values come from the encoder). The encoder runs once; the decoder advances one step at a time until [STOP]. Training = maximize the gold y's likelihood with **teacher forcing**.
- **BART** (Lewis 2019): a seq2seq denoising autoencoder. Corrupt text (token masking, deletion, **text infilling** of spans, sentence permutation, document rotation) and reconstruct the full original text autoregressively. Strong for summarization (fine-tune on article → summary).
- **BERT vs BART**: BERT = encoder only, can't generate. BART = encoder + decoder, so it can generate, and its encoder can be used wherever BERT would be.
- **T5** (Raffel 2019): span corruption. Input has gaps with sentinels ("Thank you <X> me to your party <Y> week"); output fills them ("<X> for inviting <Y> last <Z>"). C4 corpus (750GB); focus on data scale and quality; casts *every* task as text-to-text (multitask). **UnifiedQA**: one T5 handles span-extraction, abstractive, and multiple-choice QA with the format "question \n passage → answer".

Part I · Weeks 7–8 — Structured Prediction: Tagging

21. Part-of-speech tagging & sequence labeling

- **Open class** (new words join): nouns (proper: IBM, Italy; common: cat, snow), verbs, adjectives, adverbs. **Closed class**: determiners, prepositions, pronouns, conjunctions, auxiliaries, modals, particles.
- Ambiguity: "*Fed raises interest rates 0.5 percent*": Fed NNP/VBD, raises VBZ/NNS, interest NN/VBP... Penn Treebank: 44 tags.
- **Tagging as independent classification**: predict each y_i from features of position i (current word, neighbors, suffixes). Problem: predictions can be **incoherent** because nothing ties neighboring tags together. Fix with **sequence models**: HMMs and CRFs.
- Accuracy: most-frequent-tag baseline ~90%; trigram HMM ~95%; TnT 96.2%; MaxEnt 96.9%; BiLSTM-CRF/BERT ~97.5%. Remaining errors are often annotation inconsistency (Manning 2011).
- **Trigram HMM**: state = tag pair (y_{i-1}, y_i), giving more context; it's a sweet spot between capacity and data.
- FFNN taggers (Botha 2017): concatenate embeddings of nearby words plus character n-gram features. Position-sensitive, not BoW.

22. Hidden Markov Models

A **generative** model of $P(\bar{y}, \bar{x})$:

$$P(\bar{y}, \bar{x}) = P(y_1) \prod_{i=2}^{n} P(y_i \mid y_{i-1}) \prod_{i=1}^{n} P(x_i \mid y_i) \cdot P(\text{STOP} \mid y_n)$$

- Parameters: **initial** $P(y_1)$, **transitions** $P(y_i \mid y_{i-1})$ (a tag given the previous tag, not conditioned on words), **emissions** $P(x_i \mid y_i)$ (a word given its tag).
- Assumptions: tags are Markov; each word is conditionally independent of everything else given its tag.
- **Estimation** = MLE = normalized counts from labeled data. Smooth emissions (add-k) to avoid zeros for rare words.
- **Inference**: $\arg\max_{\bar{y}} P(\bar{y} \mid \bar{x}) = \arg\max_{\bar{y}} \log P(\bar{y}, \bar{x})$.

Viterbi algorithm (dynamic programming)

$$v_1(y) = \log P(y) + \log P(x_1 \mid y)$$

$$v_i(y) = \log P(x_i \mid y) + \max_{y'} \left[\log P(y \mid y') + v_{i-1}(y') \right] \quad \text{(store backpointer)}$$

$$\text{final: } \max_{y} \left[v_n(y) + \log P(\text{STOP} \mid y) \right], \text{ then follow backpointers}$$

- Runtime: $O(nk^2)$ for n words and k tags (n steps × k current tags × k previous tags). It searches implicitly over $O(k^n)$ paths. If each tag can be followed by at most l tags, there are $O(k\, l^{n-1})$ paths and Viterbi runs in $O(nkl)$.
- **Intuition**: the best path to tag y at time i must extend the best path to some y' at $i-1$, so you only keep k "best so far" scores per step.

WORKED EXAMPLE (FA22 HMM)
Initial J 2/3, N 1/3. Transitions J→{J ½, N ¼, STOP ¼}, N→{J 0, N ½, STOP ½}. Emissions J: big ½, red ¼, convertible ¼, car 0; N: convertible ⅓, car ⅔.
P(J/big N/car) = $\frac{2}{3} \cdot \frac{1}{2} \cdot \frac{1}{4} \cdot \frac{2}{3} \cdot \frac{1}{2} = \frac{1}{36}$ (don't forget STOP).
"big convertible car": big must be J and car must be N; convertible J or N → **2** valid sequences: JJN = 1/288, **JNN = 1/216** (the Viterbi answer).
"big car convertible": J N ?; N→J has probability 0, so convertible = N → only **1** sequence (JNN).

23. CRFs, forward-backward, and NER

Conditional Random Fields are discriminative (they model $P(\bar{y} \mid \bar{x})$ directly) and use arbitrary overlapping features of the whole input:

$$P(\bar{y} \mid \bar{x}) = \frac{1}{Z(\bar{x})} \exp\left(\sum_i w^\top f_e(y_i, i, \bar{x}) + \sum_i w^\top f_t(y_{i-1}, y_i)\right)$$

- Emission features can look at any word, prefix, suffix, capitalization, or gazetteer; transition features score tag bigrams. Inference: same Viterbi, with scores instead of log-probabilities.
- **Training** needs the partition function Z and the marginals $P(y_i = s \mid \bar{x})$, computed with **forward-backward**: forward α is Viterbi with **sum** instead of max; backward β goes from the end. Marginal $\propto \alpha_i(s)\beta_i(s)$. To avoid **double counting** emissions, include the emission at step i in α but not in β (β covers the next step onward).
- Gradient = gold feature counts − expected feature counts (like multiclass LR).
- **Structured (CRF) vs independent classification**: the advantage is coherent outputs (e.g., I-PER can't follow O) and joint decoding. The disadvantages are slower, more complex training and inference ($O(nk^2)$).

NER with BIO tags: B-X begins an entity of type X, I-X continues it, O is outside.
"NASA/B-ORG astronauts/O Neil/B-PER Armstrong/I-PER and/O Buzz/B-PER Aldrin/I-PER landed on the/O Moon/B-LOC".
Evaluation is **span-level**: a predicted entity counts only if both its boundaries *and* type match exactly.
Precision = correct predicted spans / predicted spans; recall = correct / gold spans; F1 = harmonic mean.
Increasing recall: add features (capitalization, gazetteers, character/subword features), use pretrained embeddings or BERT, add training data, weight the loss toward entity tags, or lower the bias toward O.

Part I · Weeks 7–8 — Structured Prediction: Parsing

24. Constituency parsing & PCFGs

- **Why syntax?** It resolves word ambiguity (Fed raises...), recovers predicate-argument structure (who did what to whom), and canonicalizes word order (SVO/SOV/VSO).
- **Constituents**: S, NP, VP, PP, ... with POS tags at the bottom. **Tests**: substitution by a proform (pronoun, "do so"), clefting ("It was with a spoon that..."), answer ellipsis.
- **Ambiguities**: PP attachment ("eat cake with icing" vs "with a spoon"), modifier scope ("plastic cup holder"), complement structure, coordination scope ("picked up his hammer and saw").

CFG $= (N, T, S, R)$: nonterminals, terminals, start symbol, rules. **Preterminals** are the POS tags (NN → cake) and form the lexicon. **PCFG**: each rule has a probability, and rules with the same parent sum to 1:

$$P(X \to \alpha) = \frac{\text{count}(X \to \alpha)}{\text{count}(X)}, \qquad P(T) = \prod_{r \in T} P(r)$$

Steps: (1) treebank of trees; (2) **binarize** (losslessly); (3) estimate rule probabilities by MLE; (4) inference $\arg\max_T P(T \mid x)$ with CKY.

> **WORKED EXAMPLE (FA22 PCFG)**
> S→PRP VP (1), VP→V NP (1), NP→J N (1), V→like (.5), J→green (.25), N→salsa (1), PRP→I (1).
> "I like green salsa" = (S (PRP I) (VP (V like) (NP (J green) (N salsa)))). P = 1·1·1·1·0.5·0.25·1 = **0.125**.
> To parse "salsa verde", add **NP → N J**. This overgenerates "I like salsa green". The fix is splitting the categories: NP→NP-EN | NP-ES; NP-EN→J-EN N; NP-ES→N J-ES.

CKY algorithm

Chart $t[i, j, X]$ = best (log) score of a derivation of symbol X over span $[i, j)$.

$$t[i, i{+}1, X] = \log P(X \to w_i) \qquad t[i, j, X] = \max_k \max_{X \to X_1 X_2} \left[\log P(X \to X_1 X_2) + t[i, k, X_1] + t[k, j, X_2]\right]$$

- Requires a **binarized** grammar (Chomsky normal form plus unary handling). Runtime $O(n^3|G|)$: spans × split points × rules.
- Answer: $t[0, n, S]$, with backpointers to recover the tree. Replace max with sum to get the inside algorithm.
- **Exam shortcut**: you rarely need to run full CKY. Check which symbols can cover each span bottom-up (e.g., "him teach I": NP over *him*, NP over *I*, VP→VBP NP over *teach I*, S→NP VP).

25. Improving PCFGs: Markovization & lexicalization

- **Independence problem**: a PCFG treats every NP the same, but subject NPs (often PRP) rewrite differently from object NPs (often DT NN or NP PP). Language isn't context-free in this probabilistic sense.
- **Vertical Markovization (parent annotation, v=2)**: relabel NP as NP^S or NP^VP. Rules become context-sensitive and often **more restrictive**. The language generated is a **subset** of (or equal to) the original's.
- **Lossless binarization**: VP→VBZ NP PP PP becomes a chain VP→VBZ VP[VBZ], VP[VBZ]→NP VP[VBZ NP], ... This is equivalent by the chain rule.
- **Horizontal Markovization** (h): remember only the last h siblings in the intermediate symbols (h=∞ is lossless). Smaller h means more generalization and sparser rules collapse together. In practice, always remember the head.
- Klein & Manning 2003 ("unlexicalized parsing"): careful annotation gets about 86 F1.

- **Lexicalization**: annotate each nonterminal with its **head word** (NP(dog), VP(ran)) using head rules (NP: rightmost noun; VP: leftmost verb...). This captures lexical preferences like PP attachment. Collins/Charniak parsers reach about 89 F1.

26. Dependency parsing

- The structure is arcs from a **head** (governor) to a **dependent** (modifier). Each word has exactly one parent; there's a ROOT; the arcs form a tree. Arc labels (nsubj, det, dobj, prep, ...) can be predicted separately with a classifier.
- **vs constituency**: for PP attachment, constituency changes several rules while dependency changes only one word's parent. Coordination is awkward in dependency, since the first conjunct is the head. **Universal Dependencies** uses one annotation scheme across many languages; dependencies are more portable cross-lingually and handle **free word order** better (that's the FA22 "alphabetized dialect" answer).
- **Projectivity**: every subtree is a contiguous span ⇔ no crossing arcs. Swiss German and free-word-order languages are often non-projective.
- Graph-based (chart) parsing: $O(n^3)$ (Eisner), tricky. **Transition-based (shift-reduce)**: greedy, left-to-right.

Arc-standard transition system

State = (stack, buffer). Start: [ROOT], [all words]. End: [ROOT], []. Operations:

- **Shift (S)**: move the top of the buffer onto the stack.
- **Left-Arc (LA)**: $\ldots w_{-2}, w_{-1} \Rightarrow \ldots w_{-1}$; w_{-2} becomes a child of w_{-1}.
- **Right-Arc (RA)**: $\ldots w_{-2}, w_{-1} \Rightarrow \ldots w_{-2}$; w_{-1} becomes a child of w_{-2}.

Exactly **2n transitions** for n words (each word is shifted once and reduced once).

WORKED EXAMPLE — "I ATE SOME SPAGHETTI BOLOGNESE"

Stack	Buffer	Action
[ROOT]	I ate some spaghetti bolognese	S
[ROOT I]	ate some ...	S
[ROOT I ate]	some spaghetti bolognese	LA (I ← ate)
[ROOT ate]	some spaghetti bolognese	S, S
[ROOT ate some spaghetti]	bolognese	LA (some ← spaghetti)
[ROOT ate spaghetti]	bolognese	S
[ROOT ate spaghetti bolognese]	—	RA (spaghetti → bolognese)
[ROOT ate spaghetti]	—	RA (ate → spaghetti)
[ROOT ate]	—	RA (ROOT → ate)

10 transitions = 2 × 5 words. You can't attach ROOT → ate early: a word must collect all its right children before it is reduced.

- **Learning**: at each state, a multiclass decision $\arg\max_{a \in \{S, LA, RA\}} w^\top f(\text{stack}, \text{buffer}, a)$. Features: top words and POS tags of stack and buffer, leftmost/rightmost children. An **oracle** converts gold trees into decision sequences (2n examples per sentence).
- **Problem**: training only sees gold states, so at test time an early mistake leads to unseen states (error propagation). Remedies: dynamic oracles, beam search.

Part II · Post-midterm topics (compact review)

These weeks (9–14) were not tested on past midterms but are in your notes. Review them for completeness or if your exam's scope is larger.

Large language models (Week 9)

- **GPT-2/3**: large left-to-right Transformer LMs. GPT-3: 175B parameters, 96 layers, 96 heads, 12k-d. Key idea: **in-context learning** — continue a pattern instead of fine-tuning.
- **Zero-shot prompting**: wrap x in a template (verbalizer), e.g., "tl;dr" for summarization. Either generate and parse, or compare label-word probabilities ("positive" vs "negative"). Results are prompt-sensitive.
- **Few-shot**: v(x₁)v(y₁)...v(x_test). Sensitive to example choice, order, and label balance. Even mislabeled demonstrations beat none (they teach format and label space). **Calibration** corrects bias toward frequent or recent labels.
- **Induction heads** (Olsson et al.): two heads in different layers. One copies previous-token info; the other finds the earlier [A] and copies the following [B], completing ...[A][B]...[A] → [B]. Their emergence correlates with in-context learning ability.
- **Instruction tuning**: supervised fine-tuning on many tasks phrased as instructions (T0, Flan-PaLM with 1800 tasks). Generalizes to new tasks given only a prompt.
- **RLHF**: humans compare two outputs → a reward model trained with Bradley-Terry ($P(y_1 \succ y_2) = \sigma(r(x, y_1) - r(x, y_2))$) → optimize the LM with PPO plus a KL penalty to stay near the base model. RL is brittle; data quality is paramount (LIMA, distillation from strong models).
- **Hallucination / factuality**: LMs model text, not facts. Check outputs by decomposing them into atomic propositions (FActScore-style), retrieving evidence, and testing entailment for each one.

Explanations & analysis (Week 10)

- Why explain: trust, fairness, causality, debugging. Local explanations (highlights), probing, text explanations.
- **LIME**: perturb the input (mask components), fit a sparse local linear model to the predictions. Model-agnostic. **Gradient-based** saliency; **integrated gradients** integrate gradients along a path from a baseline to the input (fixes saturation).
- Generated text explanations can be **plausible but unfaithful** (e-SNLI). **Probing**: train simple classifiers on frozen representations to see what's encoded.
- **Dataset artifacts**: models exploit cues such as negation words ↔ contradiction in SNLI (annotators added negation to create contradictions), and question type in SQuAD. Hypothesis-only baselines expose them.
- **Chain-of-thought**: prompt with step-by-step rationales ("Let's think step by step"). Helps large models on multi-step math and QA. Variants: program-aided reasoning (generate code), tool use (Toolformer). CoT isn't always faithful.

QA & dialogue (Week 11)

- **Reading comprehension** (SQuAD): BERT predicts start and end distributions over passage tokens, $P(\text{start} = i) = \text{softmax}(w_s^\top h_i)$; answer = best span.
- SQuAD is often easy (lexical overlap). **Adversarial distractor** sentences break models. Harder settings: **open-domain QA** (retrieve with BM25 or dense DPR, then read; Natural Questions) and **multi-hop QA** (HotpotQA, WikiHop).
- **Chatbots**: ELIZA (rule-based keyword transformation), retrieval/nearest-neighbor responses, seq2seq "translation" of utterances (bland, inconsistent persona), BlenderBot (pre-trained + fine-tuned on

PersonaChat/Empathetic Dialogues + nucleus sampling), then ChatGPT (RLHF). Hard to evaluate automatically.
- **Task-oriented dialogue**: intent classification + slot filling + API calls + generation. Data from Wizard-of-Oz collection.

MT & summarization (Week 12)

- MT difficulty: word order, morphology, ambiguity. **Word alignment** from bitext; **IBM Model 1** (alignment variables a_i, translation table $P(f \mid e)$, trained with EM).
- **Phrase-based MT**: phrase table P(f|e) + n-gram LM + distortion. Beam search where the state is which source words are covered plus the last n−1 target words.
- **Neural MT**: seq2seq with attention, then Transformers (GNMT, Transformer big/base) with BPE. **mBART** (multilingual BART pre-training) helps low-resource MT. LLMs are strong on high-resource pairs and weaker on low-resource ones.
- **Summarization**: extractive (pick sentences), compressive, or abstractive. **MMR**: choose sentences that are relevant but not redundant (TF-IDF similarity). Bigram-coverage ILP. Evaluation: **ROUGE-n** (n-gram overlap; ROUGE-2 correlates reasonably for multi-document). Neural: BART and PEGASUS (gap-sentence generation). Outputs aren't always factual; check with entailment (e.g., dependency-arc entailment).

Multilinguality, grounding, ethics (Weeks 13–14)

- **Morphology**: inflectional (arrive/arrives; je suis/tu es) vs derivational (new lexemes). Rich in Finnish and Turkish; irregular forms are common.
- **Cross-lingual transfer**: project tags across aligned bitext; direct delexicalized transfer of parsers over POS sequences. **Multilingual embeddings / mBERT** (104 Wikipedias, shared BPE): zero-shot transfer works surprisingly well, even across scripts (Urdu→Hindi), and worse for distant pairs (Japanese→English).
- **Grounding**: symbol grounding (Harnad), Chinese Room (Searle), the octopus test (Bender & Koller 2020: form ≠ meaning). **Language + vision**: captioning, VQA, **CLIP** (contrastive image-text objective).
- **Ethics**: bias amplification (imSitu cooking images are ~67% women, and models amplify this; MT gender defaults), exclusion (English/newswire-centric data), dangers ("Stochastic Parrots": environmental cost, unauditable data, no grounding; toxic degeneration), and unethical uses (misinformation, plagiarism). Remedies: datasheets for datasets, model cards, accountability.

Part III · Exam-day cheat sheet

Concept	Formula / fact
Perceptron update	On a mistake: $w \leftarrow w + y\,f(x)$ (binary, y∈{±1}). Converges iff the data are linearly separable. Watch the tie rule.
LR gradient (y=+1)	$\nabla = -f(x)(1 - P(+\|x))$; update $w \mathrel{+}= \alpha f(x)(1 - P)$. Convex.
Softmax / multiclass LR	$\nabla = -f(x, y^*) + \mathbb{E}_P[f(x, y)]$
Multiclass perceptron (DW)	$w_{y^*} \mathrel{+}= f(x)$, $w_{\hat{y}} \mathrel{-}= f(x)$
FFNN	$P(y\|x) = \text{softmax}(W g(V f(x)))$; don't initialize to zeros
Skip-gram	$P(c\|w) = \exp(v_w \cdot c_c) / \sum \exp(v_w \cdot c_{c'})$; params $2\|V\|d$; cost $O(\|V\|d)$
SGNS	$\log \sigma(w \cdot c) + \sum_k \log \sigma(-w \cdot c_k)$; ≈ factorizing PMI − log k
Cosine	$a \cdot b / (\|a\|\|b\|)$ — ignores length; dot product favors long (frequent) vectors
Bigram MLE	$P(b\|a) = c(a, b)/c(a)$ (divide by count of the context word)
Perplexity	$\exp(-\frac{1}{n}\sum \log P)$ = geometric mean of 1/P; uniform over V → $\|V\|$
Self-attention	$\text{softmax}(QK^\top/\sqrt{d_k})V$; A is n×n; cost $O(n^2 d)$, $O(1)$ sequential steps
Transformer params	≈12d² per layer (4d² attention + 8d² FFN); BERT-base 110M, large 340M
LM output shape	[batch, seq, vocab]; attention [batch, seq, seq]; causal mask = −∞ above the diagonal
Decoding	Greedy, beam (MT), sampling (9 = $\|V\|^T$ options), nucleus (smallest set with mass ≥ p)
BERT	MLM 15% (80/10/10) + NSP; [CLS]; bidirectional; can't generate
BART / T5	Seq2seq denoising (infilling, permutation…) / span corruption with sentinels; text-to-text
HMM	$P(y_1)\prod P(y_i\|y_{i-1})\prod P(x_i\|y_i)\,P(\text{STOP}\|y_n)$
Viterbi	$O(nk^2)$ time, searches k^n paths; with fan-out l: $k\,l^{n-1}$ paths
CRF	Discriminative; forward-backward for marginals; gradient = gold − expected features
NER eval	Span-level exact match of boundaries + type; P = correct/predicted, R = correct/gold
PCFG	$P(X \to \alpha) = c(X \to \alpha)/c(X)$; P(tree) = product of rules
CKY	$O(n^3\|G\|)$, binarized grammar, chart t[i,j,X]
Vertical Markovization	NP → NP^parent; the generated language is a subset of the original's
Arc-standard	S, LA (top-2 ← top), RA (top-2 → top); 2n transitions

STRATEGY TIPS FOR THIS EXAM

- "Select all that apply" gets partial credit, so reason about each option independently. "Always" and "guaranteed" are usually false (perceptron convergence).
- Hand computations: write every intermediate step (partial credit). Check whether STOP probabilities are included, whether ties count as positive, and whether UNK mapping applies.
- Parameter-count questions: count FFNs and attention blocks; positional sinusoids are free.
- Short answers: 1–2 precise sentences. The instructions warn that long, incorrect answers can be penalized.
- It's open book: bookmark this guide's cheat sheet, the BERT/Transformer slides, and the HMM/PCFG worked examples.